



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE24CS151B**

***Lecture #7
Creation and Usage of Header files***

**By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: File Handling and Portable Programming****Topic: Creation and Usage of Header files**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

In C programming, header files serve as a way to organize code for better **modularity, readability, and reusability**. A header file typically contains **function declarations, macro definitions, constants, and structure or type definitions that are used across multiple source files**. C provides several **predefined header files** like `<stdio.h>`, `<math.h>`, and also allows you to create **custom header files** to include user-defined functions and types. **Header files are included in the program using the `#include` directive**.

Advantages of using header files:

- Avoiding rewriting the code across different .c files.
- Separating declarations(interface) from definitions(implementation), keeping code clean and organized.
- Easier maintenance and collaboration in large projects.

Let us consider an example code related to **finding whether the number is a palindrome or not**.

Coding Example_1: All operations are in one single file.

```
#include<stdio.h>
int main()
{
    int n;
    printf("Enter the number\n");
    scanf("%d", &n);
    int rev = 0;    int number = n;
    while(n>0) // can be just while(n).
    {
        rev = rev * 10 + n % 10; n /= 10;
    }
    if(number == rev)
        printf("%d is palindrome\n",number);
    else
        printf("%d is not palindrome\n",number);
    return 0;
}
```

```
C:\Users\Dell>a
Enter the number
-111
-111 is not palindrome

C:\Users\Dell>a
Enter the number
12121
12121 is palindrome

C:\Users\Dell>a
Enter the number
1
1 is palindrome
```

Coding Example_2: Separating the interface and implementation - Function to check whether the given number is palindrome or not.

```
include<stdio.h>
int is_palin(int); // interface
int main(){
    int n;
    printf("Enter the number\n");
    scanf("%d", &n);
    if(is_palin(n))
        printf("%d is a palindrome\n", n);
    else
        printf("%d is not a palindrome\n", n);
}
int is_palin(int n) // implementation
{
    int rev = 0;
    int number = n;
    while(n>0){
        rev = rev * 10 + n % 10; n /= 10;
    }
    return number == rev;
}
```

```
C:\Users\Dell>a
Enter the number
123
123 is not a palindrome

C:\Users\Dell>a
Enter the number
-3443
-3443 is not a palindrome

C:\Users\Dell>a
Enter the number
15351
15351 is a palindrome
```

Now let us create separate files for interface and implementation to follow modular programming practices and enable code reuse and maintainability.

Coding Example_3: Creation of header file. Separating the client and server.

//palin.h

```
int is_palin(int); // interface
```

//client code – palin_client.c

```
#include<stdio.h>
```

```
int main(){
    int n;
    printf("Enter the number\n"); scanf("%d", &n);
    if(is_palin(n))
        printf("%d is a palindrome\n", n);
    else
        printf("%d is not a palindrome\n", n);
}
```

```
C:\Users\Dell>gcc -c palin_client.c
C:\Users\Dell>gcc -c palin.c
C:\Users\Dell>gcc palin_client.o palin.o
C:\Users\Dell>a.exe
Enter the number
121
121 is a palindrome

C:\Users\Dell>a.exe
Enter the number
-121
-121 is not a palindrome

C:\Users\Dell>a.exe
Enter the number
5
5 is a palindrome
```

//Server code Implementation – palin.c

```
int is_palin(int n) // implementation
{
    int rev = 0;
    int number = n;
    while(n>0){
        rev = rev * 10 + n % 10; n /= 10;
    }
    return number == rev;
}
```

Execution Steps when using Header and Implementation Files in C.

1. Compile Each .c File into an Object File (.o)
2. Link All .o Files to Create the Final Executable
3. Run the Executable

For the above codes,

gcc -c palin.c palin_client.c // results in 2 .o files

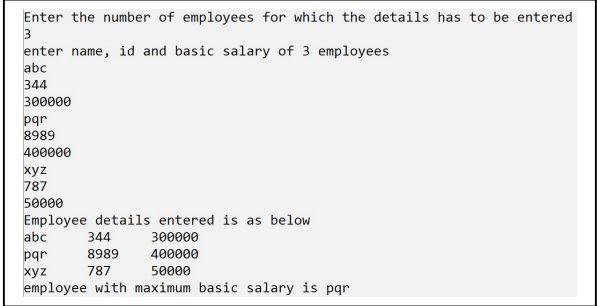
gcc palin.o palin_client.o // results in an executable file

a.exe OR ./aout // Execute the executable

Let us consider an application in C to read and display the details of n Employees. Separate the interface and implementation.

Coding Example_4: Employee application segregating client and server and display the employee details whose salary is max.

```
//employee.h
typedef struct employee
{
    char name[20];
    int id;
    int basic_salary;
    int total_salary;
}employee_t;
void scan_details(employee_t*, int);
void display_details(const employee_t*, int);
int calculate_total_salary(employee_t*, int,int);
employee_t max(employee_t*, int);
```



```
Enter the number of employees for which the details has to be entered
3
Enter name, id and basic salary of 3 employees
abc
344
300000
pqr
8989
400000
xyz
787
50000
Employee details entered is as below
abc      344      300000
pqr      8989      400000
xyz      787      50000
employee with maximum basic salary is pqr
```

```
// employee_server.c
```

```
#include "employee.h"
```

```
#include <stdio.h>
```

```
void scan_details(employee_t* e, int n){  
    for(int i = 0; i < n; i++)  
        scanf("%s %d %d", (e+i)->name, &(e+i)->id, &(e+i)->basic_salary);  
}
```

```
void display_details(const employee_t* e, int n){  
    for(int i = 0; i < n; i++)  
        printf("%s\t%d\t%d\n", (e+i)->name, (e+i)->id, (e+i)->basic_salary);  
}
```

```
employee_t max(employee_t* e, int n){  
    int i;    int max = 0;    employee_t e_max;  
    for(i = 0; i < n; i++){  
        if((e+i)->basic_salary > max) {  
            max = (e+i)->basic_salary;        e_max = *(e+i);  
        }  
    }  
    return e_max;  
}
```

```
//employee_client.c
```

```
#include <stdio.h>
```

```
#include "employee.h"
```

```
int main()  
{  
    employee_t e[1000]; int n, id;  
    printf("Enter the number of employees for which the details has to be entered\n");  
    scanf("%d", &n);  
    printf("enter name, id and basic salary of %d employees\n", n);  
    scan_details(e, n);  
    printf("Employee details entered is as below\n");  
    display_details(e, n);  
    employee_t e_max = max(e, n);  
    printf("employee with maximum basic salary is %s", e_max.name);  
    return 0;  
}
```

Keep your code neat — headers to the rescue!!